

Curso: Programación Avanzada

Guía: 17

Título: MAQUETACIÓN CON PRIMENG

Duración estimada: 240 minutos

Docentes: Christian Andrés Candela

Objetivo

Integrar PrimeNG 21 en el proyecto Angular de solicitudes universitarias para estilizar el formulario de login y el listado de solicitudes con componentes de UI de alto nivel, comprendiendo el sistema de temas basado en Design Tokens y la arquitectura de componentes standalone de PrimeNG.



Conceptos Básicos Previos

- **Angular Standalone Components:** Componentes que no requieren `NgModule` ; estándar desde Angular 17.
- **PrimeNG:** Biblioteca de componentes UI open-source para Angular con más de 80 componentes accesibles y personalizables.
- **Sistema de Temas PrimeNG v21:** Sistema de diseño basado en **Design Tokens** y **Presets** (Aura , Material , Lara , Nora). Reemplaza el sistema de temas CSS de versiones anteriores.
- **Styled Mode vs Unstyled Mode:** PrimeNG puede operar con sus estilos integrados (Styled) o sin ningún estilo (Unstyled), para integración con frameworks CSS externos como Tailwind.
- **Reactive Forms + PrimeNG:** Los componentes PrimeNG son compatibles con `formControlName` y `[formGroup]` sin configuración adicional.
- **Signals:** `signal()` , `computed()` , `toSignal()` — patrón de Guías 15 y 16, que se mantiene aquí.

Contextualización Teórica

◆ ¿Qué es PrimeNG?

PrimeNG, desarrollado por PrimeTek, es la biblioteca de componentes UI más popular del ecosistema Angular. Ofrece componentes accesibles y altamente configurables listos para producción: formularios, tablas de datos, diálogos, menús, gráficos y más.

Versión actual: PrimeNG **21.1.6** (compatible con Angular 21).

Documentación oficial: <https://primeng.org/>

◆ Bootstrap vs. PrimeNG — ¿Cuándo usar cada uno?

La Guía 16 enseñó Bootstrap 5. Esta guía enseña PrimeNG. Ambas son válidas; la elección depende del tipo de proyecto:

Criterio	Bootstrap 5	PrimeNG 21
Orientación	Framework CSS general	Biblioteca de componentes Angular
Componentes complejos	Básicos (navbar, card, badge)	Avanzados (DataTable, Calendar, FileUpload, AutoComplete)
Integración con Angular	Manual (clases CSS en HTML)	Nativa (componentes Angular con @Input , @Output)
Formularios reactivos	Clases CSS + lógica TypeScript propia	<code>formControlName</code> nativo en cada componente
Sistema de temas	Variables CSS + Sass	Design Tokens + Presets
Curva de aprendizaje	Baja	Media
Ideal para	Sitios informativos, layouts generales	Aplicaciones de gestión (CRUD, dashboards, backoffice)

En proyectos reales es común combinar ambas: Bootstrap para el layout general (grid, navbar, footer) y PrimeNG para los componentes de datos complejos (tablas, formularios avanzados).

◆ Sistema de Temas PrimeNG v21

Desde PrimeNG v19, el sistema de temas fue completamente rediseñado alrededor de **Design Tokens** y **Presets**.

¿Qué es un Design Token? Es una variable con nombre que representa una decisión de diseño (color, tamaño, radio, sombra). En lugar de hardcodear `#3b82f6`, se define `--p-primary-500: #3b82f6` y todos los componentes lo consumen.

Presets disponibles en v21:

Preset	Estilo visual
Aura	Moderno, bordes suaves, elevado
Material	Google Material Design 3
Lara	Elegante, profesional
Nora	Minimalista, oscuro

La configuración se hace en `app.config.ts` con `providePrimeNG`:

```
import { Aura } from 'primeng/themes/aura';

providePrimeNG({
  theme: {
    preset: Aura,
    options: {
      darkModeSelector: '.app-dark' // clase CSS para activar dark mode
    }
  }
})
```

◆ Arquitectura Standalone — Componentes Directos sin Módulos

En PrimeNG v19+, **todos los componentes son standalone**. Esto significa:

- **No** se importan módulos (`InputTextModule`, `ButtonModule` ...) — eso era el patrón de PrimeNG v17 y anteriores.
- Se importa directamente la clase del **componente** o la **directiva**:

```
// ✅ PrimeNG v21 — componentes/directivas directas
import { InputText } from 'primeng/inputtext';
import { Password } from 'primeng/password';
import { Button } from 'primeng/button';
import { Message } from 'primeng/message';
import { Card } from 'primeng/card';
import { Tag } from 'primeng/tag';

@Component({
  imports: [InputText, Password, Button, Message, ReactiveFormsModule],
})
```

En versiones anteriores (v17-v18) se usaba `InputTextModule` , `ButtonModule` , etc. Si ves esa sintaxis en tutoriales online, probablemente estén desactualizados.

◆ El componente `p-fluid` — Inputs a ancho completo

PrimeNG v19 introdujo el componente `p-fluid` como contenedor que hace que todos los inputs y botones hijos ocupen el **100% del ancho** automáticamente, sin necesidad de `width: 100%` en CSS:

```
<p-fluid>
  <input pInputText formControlName="username" />
  <!-- El input ocupa el 100% del contenedor automáticamente -->
</p-fluid>
```

⚠ Precauciones y Recomendaciones

1. **PrimeNG v21 requiere** `@angular/cdk` como dependencia de pares — se instala junto con PrimeNG.
2. **No uses módulos legacy** (`InputTextModule` , `ButtonModule`) — usa los componentes directos.
3. **Reinicia el servidor** (`npm start`) tras modificar `app.config.ts` o instalar nuevas dependencias.
4. **Consulta la documentación oficial** para cada componente: <https://primeng.org/>.
5. El proyecto ya tiene Bootstrap instalado (Guía 16). PrimeNG y Bootstrap pueden coexistir; los estilos de PrimeNG tienen scope propio gracias a los Design Tokens.



Artefactos Requeridos

- Proyecto Angular de la Guía 16 funcionando (con Bootstrap, Header, Footer y `ListaSolicitudes`).
- Backend Spring Boot corriendo en `http://localhost:8080` .
- Node.js y Angular CLI instalados.



Evaluación o Resultado Esperado

Se espera que el estudiante:

1. Instale y configure PrimeNG 21 con el preset `Aura` en el proyecto.
2. Estilice el formulario de Login usando `pInputText` , `p-password` , `p-button` y `p-message` .
3. Aplique `p-card` y `p-tag` en el listado de solicitudes.
4. Mantenga todos los patrones Angular 21 establecidos en Guías 15 y 16 (`signal` , `computed` , `takeUntilDestroyed`).

Procedimiento

1. Configuración Inicial

1. Instala PrimeNG y su dependencia de pares:

```
npm install primeng @angular/cdk
```

`@angular/cdk` (Angular Component Dev Kit) es una **dependencia requerida** de PrimeNG v21.
No se instala automáticamente.

Verificación: Confirma que `primeng` y `@angular/cdk` aparecen en `package.json` .

2. Configura PrimeNG en `src/app/app.config.ts` :

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { providePrimeNG } from 'primeng/config';
import { Aura } from 'primeng/themes/aura';
import { routes } from './app.routes';
import { authInterceptor } from './interceptores/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])),
    provideAnimationsAsync(),
    providePrimeNG({
      theme: {
        preset: Aura,
        options: {
          darkModeSelector: '.app-dark'
        }
      }
    })
  ]
};
```

¿Por qué `provideAnimationsAsync` ? PrimeNG usa animaciones para transiciones (diálogos, overlays, paneles). `provideAnimationsAsync` carga el módulo de animaciones de forma asíncrona, mejorando el tiempo de carga inicial.

3. Ejecuta la aplicación y verifica que no hay errores:

```
npm start
```

2. 🌐 Verificar las Rutas del Proyecto

Asegúrate de que `app.routes.ts` refleje el estado actual del proyecto (sin el componente Registro eliminado en Guía 16):

```
import { Routes } from '@angular/router';
import { Login }      from './componentes/login/login';
import { NuevaSolicitud } from './componentes/nueva-solicitud/nueva-solicitud';
import { ListaSolicitudes } from './componentes/lista-solicitudes/lista-solicitudes';

export const routes: Routes = [
  { path: '', component: /* tu componente Inicio */ },
  { path: 'login', component: Login },
  { path: 'nueva-solicitud', component: NuevaSolicitud },
  { path: 'lista-solicitudes', component: ListaSolicitudes },
  { path: '**', pathMatch: 'full', redirectTo: '/' },
];
```

3. 🔑 Login con PrimeNG

Vamos a reemplazar el HTML de Bootstrap del Login (Guía 16) por componentes PrimeNG, manteniendo el TypeScript de Angular 21 sin cambios.

3.1 Actualiza los imports en `login.ts`

Añade los componentes PrimeNG al arreglo `imports` del componente:

```

import { Component, computed, DestroyRef, inject, signal } from '@angular/core';
import { takeUntilDestroyed, toSignal } from '@angular/core/rxjs-interop';
import { FormBuilder, ReactiveFormsModule, Validators } from '@angular/forms';
import { Router } from '@angular/router';

// PrimeNG v21 – componentes directos (sin módulos)
import { InputText } from 'primeng/inputtext';
import { Password } from 'primeng/password';
import { Button } from 'primeng/button';
import { Message } from 'primeng/message';
import { IftaLabel } from 'primeng/iftalabel';
import { Fluid } from 'primeng/fluid';

import { AuthService } from '../servicios/auth';

@Component({
  selector: 'app-login',
  imports: [
    ReactiveFormsModule,
    InputText, Password, Button, Message, IftaLabel, Fluid
  ],
  templateUrl: './login.html',
  styleUrls: ['./login.css']
})
export class Login {
  private authService = inject(AuthService);
  private router = inject(Router);
  private destroyRef = inject(DestroyRef);

  // ✅ Inicialización directa – sin constructor()
  loginForm = inject(FormBuilder).group({
    username: ['', [Validators.required, Validators.email]],
    password: ['', [Validators.required, Validators.minLength(6)]]
  });

  // ✅ Signals de estado de UI
  isLoading = signal(false);
  result = signal('');

  // ✅ toSignal() convierte el Observable del form en un Signal
  private formStatus = toSignal(
    this.loginForm.statusChanges,
    { initialValue: 'INVALID' as const }
  );

```

```

);

// ✅ computed(): el botón solo se habilita si el form es válido y no está cargando
canSubmit = computed(() => this.formStatus() === 'VALID' && !this.isLoading());

onSubmit(): void {
  if (this.loginForm.invalid) return;

  const { username, password } = this.loginForm.value as { username: string; password: string };
  this.isLoading.set(true);
  this.result.set('');

  this.authService.login({ username, password })
    .pipe(takeUntilDestroyed(this.destroyRef))
    .subscribe({
      next: () => {
        this.isLoading.set(false);
        this.authService.isAuthenticated.set(true);
        this.router.navigate(['/lista-solicitudes']);
      },
      error: (err: Error) => {
        this.isLoading.set(false);
        this.result.set(err.message || 'Error al iniciar sesión');
      }
    });
}
}

```

Conceptos clave del TypeScript:

- `IftaLabel` (In-the-Field Top Aligned Label): Componente PrimeNG que posiciona el label **dentro** del campo de texto, moviéndolo arriba cuando el usuario escribe (patrón Material Design).
- `Fluid` : Envuelve los campos y botones para que ocupen el **100% del ancho** automáticamente.
- Los signals `isLoading` , `result` , `canSubmit` son idénticos al patrón de Guía 15.

3.2 Reemplaza login.html

```
<div class="login-wrapper">
  <p-card class="login-card">

    <!-- Cabecera -->
    <div class="login-header">
      <i class="pi pi-lock text-primary" style="font-size: 2rem"></i>
      <h2 class="mt-2">Iniciar Sesión</h2>
      <p class="text-secondary">Sistema de Solicitudes UQ</p>
    </div>

    <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
      <p-fluid>
        <div class="flex flex-col gap-4">

          <!-- Campo correo con IftaLabel -->
          <p-iftalabel>
            <input pInputText id="username"
              type="email"
              formControlName="username"
              autocomplete="username" />
            <label for="username">Correo electrónico</label>
          </p-iftalabel>
          @if (loginForm.get('username')?.invalid && loginForm.get('username')?.touched
            <small class="p-error">
              Ingresa un correo electrónico válido.
            </small>
          }

          <!-- Campo contraseña -->
          <p-iftalabel>
            <p-password id="password"
              formControlName="password"
              [feedback]="false"
              [toggleMask]="true" />
            <label for="password">Contraseña</label>
          </p-iftalabel>
          @if (loginForm.get('password')?.invalid && loginForm.get('password')?.touched
            <small class="p-error">
              La contraseña debe tener al menos 6 caracteres.
            </small>
          }
        </div>
      </p-fluid>
    </form>
  </p-card>
</div>
```

```

        <!-- Mensaje de error del servidor -->
        @if (result()) {
            <p-message severity="error" [text]="result()" />
        }

        <!-- Botón con loading state -->
        <p-button
            type="submit"
            label="Ingresar"
            severity="primary"
            [disabled]="!canSubmit()"
            [loading]="isLoading()"
        />

    </div>
</p-fluid>
</form>

</p-card>
</div>

```

Componentes PrimeNG usados:

- `pInputText` (directiva): Aplica estilo PrimeNG a un `<input>` nativo.
- `p-password` : Campo de contraseña con botón de mostrar/ocultar (`[toggleMask]`) y sin indicador de fortaleza (`[feedback]="false"`).
- `p-iftalabel` : Label que flota al escribir (In-the-Field Top Aligned).
- `p-button` : Botón con `severity` para variantes de color y `[loading]` para spinner automático.
- `p-message` : Mensaje de alerta inline. `severity` acepta: `success` , `info` , `warn` , `error` .
- `p-fluid` : Hace que inputs y botones ocupen el 100% del ancho.

3.3 Actualiza login.css

```
.login-wrapper {
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 80vh;
  padding: 1rem;
}

.login-card {
  width: 100%;
  max-width: 420px;
}

.login-header {
  text-align: center;
  margin-bottom: 1.5rem;
}

.login-header h2 {
  font-size: 1.5rem;
  font-weight: 600;
  margin: 0.5rem 0 0.25rem;
}

/* Necesario para que p-card use la importación de Card */
:host {
  display: block;
}
```

Nota: Para que p-card funcione, añade Card al arreglo imports del componente:

```
import { Card } from 'primeng/card';
@Component({ imports: [..., Card] })
```

3.4 Ejecuta y prueba el formulario:

```
npm start
```

1. Navega a <http://localhost:4200/login>.

2. Haz click en el campo email — el label debe desplazarse hacia arriba (efecto IftaLabel).
3. Intenta enviar con campos vacíos — el botón debe estar deshabilitado.
4. Ingresa un email inválido y toca el campo — el mensaje de error debe aparecer.
5. Con credenciales correctas, el botón debe mostrar un spinner y redirigir a `/lista-solicitudes` .

4. Actualizar el AuthService

El AuthService debe estar sincronizado con los patrones de Guías 15 y 16. Aplica estos cambios:

src/app/servicios/auth.ts — versión completa:

```
import { Injectable, inject, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, tap, catchError, throwError } from 'rxjs';
import { LoginRequest } from '../dto/login-request';
import { TokenResponse } from '../dto/token-response';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private http = inject(HttpClient);
  private readonly API = 'http://localhost:8080/api/auth';

  // ✅ Signal de autenticación – fuente de verdad única para toda la app
  // El Header de Guía 16 lo lee; Login de Guía 17 lo actualiza
  isAuthenticated = signal(!localStorage.getItem('token'));

  login(credenciales: LoginRequest): Observable<TokenResponse> {
    return this.http.post<TokenResponse>(`${this.API}/login`, credenciales).pipe(
      tap(response => {
        // ✅ Clave 'token' – consistente con el AuthInterceptor de Guía 15
        localStorage.setItem('token', response.token);
        localStorage.setItem('roles', JSON.stringify(response.roles));
      }),
      catchError(error => {
        const mensaje = error.error?.message || 'Credenciales inválidas';
        return throwError(() => new Error(mensaje));
      })
    );
  }

  getToken(): string | null {
    return localStorage.getItem('token');
  }

  logout(): void {
    localStorage.removeItem('token');
    localStorage.removeItem('roles');
    this.isAuthenticated.set(false);
  }
}
```

Cambios clave respecto a la versión original de esta guía:

Cambio	Razón
<code>localStorage.setItem('token', ...)</code>	Consistente con el <code>AuthInterceptor</code> de Guía 15 (usaba clave <code>'token'</code>)
<code>isAuthenticated = signal(...)</code>	El <code>Header</code> de Guía 16 lo lee como <code>signal</code> reactivo
<code>isAuthenticated.set(false)</code> en <code>logout()</code>	El <code>Header</code> se actualiza automáticamente sin recargar la página

5. Lista de Solicitudes con PrimeNG

Ahora aplicamos PrimeNG al componente `ListaSolicitudes` de Guía 16, reemplazando `badges` y `cards` de Bootstrap por `p-tag` y `p-card` . Esto ilustra cómo **migrar gradualmente** un componente de Bootstrap a PrimeNG.

5.1 Actualiza lista-solicitudes.ts — añade imports PrimeNG

```
import { Component, inject } from '@angular/core';
import { toSignal } from '@angular/core/rxjs-interop';
import { RouterLink } from '@angular/router';
import { SlicePipe } from '@angular/common';
import { Card } from 'primeng/card';
import { Tag } from 'primeng/tag';
import { SolicitudesService } from '../../servicios/solicitudes';
import { SolicitudResumen } from '../../modelos/solicitudes';

@Component({
  selector: 'app-lista-solicitudes',
  imports: [RouterLink, SlicePipe, Card, Tag],
  templateUrl: './lista-solicitudes.html',
  styleUrls: ['./lista-solicitudes.css']
})
export class ListaSolicitudes {
  private solicitudesService = inject(SolicitudesService);

  solicitudes = toSignal(
    this.solicitudesService.listar(),
    { initialValue: [] as SolicitudResumen[] }
  );

  /**
   * Convierte el estado en un severity de p-tag.
   * Union type garantiza en compilación que solo se usen valores válidos.
   */
  tagSeveridad(estado: string): 'success' | 'info' | 'warn' | 'danger' | 'secondary' |
    const mapa: Record<string, 'success' | 'info' | 'warn' | 'danger' | 'secondary' | '
    REGISTRADA: 'secondary',
    CLASIFICADA: 'info',
    EN_ATENCION: 'contrast',
    ATENDIDA: 'success',
    CERRADA: 'secondary',
    CANCELADA: 'danger',
  };
  return mapa[estado] ?? 'secondary';
}

tagPrioridad(prioridad: string): 'success' | 'warn' | 'danger' {
  const mapa: Record<string, 'success' | 'warn' | 'danger'> = {
```

```
    BAJA: 'success', MEDIA: 'warn', ALTA: 'danger',  
  };  
  return mapa[prioridad] ?? 'success';  
}  
}
```

¿Por qué **severity tipado**? A diferencia del badge Bootstrap (string libre), `p-tag` usa un union type TypeScript. Esto ofrece **autocompletado** en el editor y **errores en compilación** si el valor no es válido — ventaja clave de los componentes Angular sobre las clases CSS.

5.2 Actualiza lista-solicitudes.html

```
<div class="container mt-4">

  <div class="d-flex justify-content-between align-items-center mb-4">
    <h1 id="titulo-listado" class="h3 mb-0">
      <i class="pi pi-list-check me-2"></i>Mis Solicitudes
    </h1>
    <a routerLink="/nueva-solicitud" class="btn btn-primary">
      <i class="pi pi-plus me-1"></i>Nueva Solicitud
    </a>
  </div>

  @if (solicitudes().length === 0) {
    <div class="text-center py-5 text-muted">
      <i class="pi pi-inbox" style="font-size: 3rem"></i>
      <p class="h5 mt-3">No tienes solicitudes registradas aún.</p>
    </div>
  } @else {
    <div class="row g-3">
      @for (solicitud of solicitudes(); track solicitud.id) {
        <div class="col-md-6 col-lg-4">
          <p-card class="h-100" [class.prioridad-alta]="solicitud.prioridad === 'ALTA'"

            <ng-template pTemplate="header">
              <div class="d-flex justify-content-between align-items-center px-3 pt-3">
                <span class="fw-semibold">{{ solicitud.codigo }}</span>
                <p-tag [value]="solicitud.estado"
                  [severity]="tagSeveridad(solicitud.estado)" />
              </div>
            </ng-template>

            <ng-template pTemplate="content">
              <p class="text-muted small mb-1">
                <i class="pi pi-tag me-1"></i>{{ solicitud.tipo }}
              </p>
              <p class="text-muted small mb-0">
                <i class="pi pi-calendar me-1"></i>
                {{ solicitud.fechaCreacion | slice:0:10 }}
              </p>
            </ng-template>

            <ng-template pTemplate="footer">
```

```

        <div class="d-flex justify-content-between align-items-center">
            <p-tag [value]="solicitud.prioridad"
                [severity]="tagPrioridad(solicitud.prioridad)"
                icon="pi pi-arrow-up" />
            <small class="text-muted">{{ solicitud.nombrePropietario }}</small>
        </div>
    </ng-template>

    </p-card>
</div>
}
</div>
}
</div>

```

ng-template pTemplate : `p-card` usa proyección de contenido nombrada. Los slots disponibles son: `header` , `title` , `subtitle` , `content` , `footer` . Esto permite personalizar cada sección del componente de forma declarativa, un patrón común en PrimeNG para componentes complejos.

PrimeIcons (pi pi-*): PrimeNG incluye su propia biblioteca de iconos `PrimeIcons` , instalada automáticamente con `primeng` . Es el equivalente a Font Awesome dentro del ecosistema PrimeNG.

5.3 Ejecuta y compara

```
npm start
```

Inicia sesión y navega a **Mis Solicitudes**. Compara con la versión Bootstrap de Guía 16:

- `p-tag` usa colores del preset Aura con bordes redondeados semánticos.
- `p-card` tiene estilos y sombras propias del tema.
- Los iconos usan PrimeIcons (`pi pi-*`) en lugar de Font Awesome (`fa-solid fa-*`).

6. 📦 Control de Versiones

```
git add .  
git commit -m "feat: integración PrimeNG 21 – login y lista de solicitudes"  
git push origin main
```

🔍 Solución de Problemas

- **Module not found: 'primeng/themes/aura' :**
 - Verifica que instalaste PrimeNG v21: `npm install primeng @angular/cdk` .
 - En versiones anteriores (v19-v20) el import era `@primeng/themes/aura` . En v21 es `primeng/themes/aura` .
- **NG0303: Can't bind to 'formControlName' :**
 - Falta `ReactiveFormsModule` en el arreglo `imports` del componente.
- **Los estilos PrimeNG no se aplican (inputs sin estilos):**
 - Confirma que `providePrimeNG(...)` está configurado en `app.config.ts` .
 - Asegúrate de importar el componente directo (`InputText` , no `InputTextModule`).
 - Reinicia el servidor (`Ctrl+C` + `npm start`) después de modificar `app.config.ts` .
- **El botón no muestra el spinner de `[loading]` :**
 - Verifica que `[loading]="isLoading()"` incluye los paréntesis del signal.
- **p-card no renderiza correctamente:**
 - Añade `Card` al arreglo `imports` del componente.
 - Añade `:host { display: block; }` en el CSS del componente.
- **El Header no se actualiza tras el login:**
 - Verifica que `onSubmit()` llama a `this.authService.isAuthenticated.set(true)` tras el login exitoso.
 - Verifica que el Header usa `isLoggedIn = this.authService.isAuthenticated` (referencia al signal, no copia).

📖 Para la Próxima Clase

1. Investiga sobre **Guards de ruta** en Angular para proteger `/lista-solicitudes` con el signal `isAuthenticated` :

- Referencia: [Angular Route Guards](#)
2. Explora `p-table` para mostrar solicitudes en tabla con paginación, ordenamiento y filtros:
 - Referencia: [PrimeNG Table](#)
 3. Explora `p-toast` como alternativa a `p-message` para notificaciones no intrusivas:
 - Referencia: [PrimeNG Toast](#)

Preguntas para Reflexión

- ¿Qué ventaja ofrece `p-tag` con `severity` tipado sobre el badge CSS de Bootstrap?
- ¿Por qué en PrimeNG v21 se importan componentes directos en lugar de módulos?
- ¿Qué es un Design Token y cómo diferencia el sistema de temas de PrimeNG v21 del de versiones anteriores?
- ¿En qué tipo de proyecto elegirías PrimeNG sobre Bootstrap, y viceversa?
- ¿Por qué es importante que `AuthService.isAuthenticated` sea un `signal` y no un método que retorna `boolean` ?

Referencias Bibliográficas

- [PrimeNG 21 — Documentación oficial](#)
- [PrimeNG — Instalación](#)
- [PrimeNG — Sistema de temas](#)
- [PrimeNG — Password](#)
- [PrimeNG — Button](#)
- [PrimeNG — Card](#)
- [PrimeNG — Tag](#)
- [Angular toSignal](#)
- [Angular Reactive Forms](#)
- [Angular Route Guards](#)